

APMA 4302: Methods in Computational Science

Spring 2026

Marc Spiegelman (APAM/DEES)

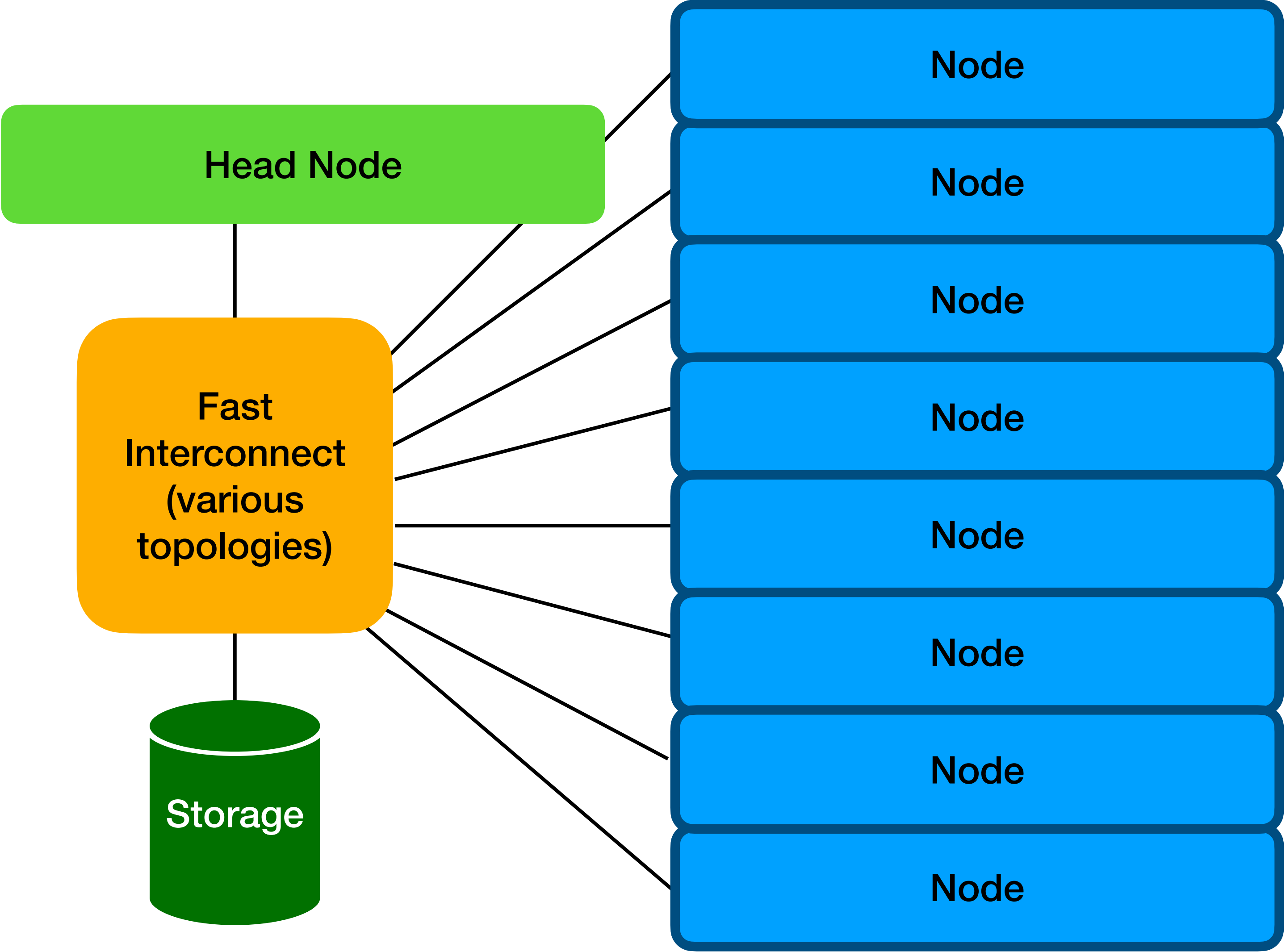
Lecture 21

Working on HPC Clusters: Environment and Performance

Outline

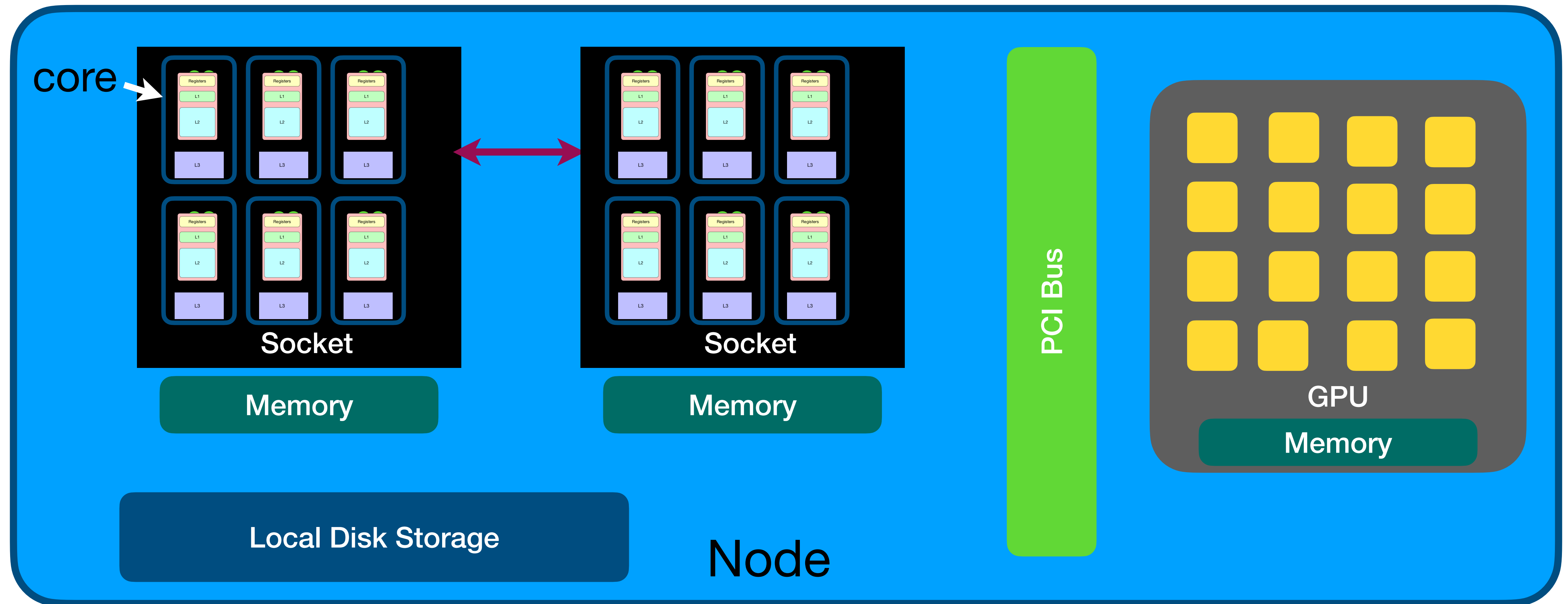
- Where We Are
- From Laptops to HPC clusters
 - General Architectures
 - Standard Toolset: git, vscode, modules, *containers* (docker/apptainer)
 - Job submission: Slurm
 - file-transfer/visualization: globus, paraview
- Performance and Scaling:
 - Streams benchmark
 - Strong Scaling v. Weak Scaling v. Static Scaling
 - Results: Ch 8 and firedrake examples on Ginsburg

Generic Cluster Architecture



Typical Compute Node

mid 2010' dual socket, shared memory, multi-core Node



Available Architectures at Columbia

(<https://www.cuit.columbia.edu/shared-research-computing-facility>)

Ginsburg:

Hardware

- Ginsburg has 286 nodes with a total of 9,152 cores (32 cores per node)
- 191 Standard Nodes (192 GB)
 - 56 High Memory Nodes (768 GB)
 - 18 GPU Nodes with two Nvidia RTX 8000 GPU modules
 - 4 GPU Nodes with two Nvidia V100S GPU modules
 - 9 GPU Nodes with two Nvidia A40 GPU modules
 - 8 GPU nodes with two Nvidia A100 GPU modules

Standard Nodes

Ginsburg has 191 Standard Nodes with the following specifications:

Model	Dell
CPU	Intel Xeon Gold 6226 2.9 Ghz
Number of CPUs	2
Cores per CPU	16
Total Cores	32
Memory	192 GB
Network	HDR Infiniband

Insomnia:

Hardware

- Insomnia has 90 nodes with a cluster total of 14,288 cores (80 physical cores per node, doubled via hyperthreading) running Red Hat Enterprise Linux 9.3
- 41 Standard Nodes (512 GB ram)
 - 19 High Memory Nodes (1 TB ram)
 - 13 GPU Nodes with eight NVIDIA A600 GPU modules
 - 2 GPU Nodes with four NVIDIA A600 GPU modules
 - 3 GPU Nodes with two NVIDIA H100 GPU modules
 - 3 GPU Nodes with two NVIDIA L40 GPU modules
 - 9 GPU Nodes with two NVIDIA L40s GPU modules

Standard Nodes

Model	Dell
CPU	Intel Xeon Platinum 8460Y 2 Ghz
Number of CPUs	2
Cores per CPU	40
Total PHYSICAL Cores per node	80
Total USABLE Cores per node	160 (hyperthreading is enabled on Insomnia)
Memory	512 GB
Network	HDR Infiniband

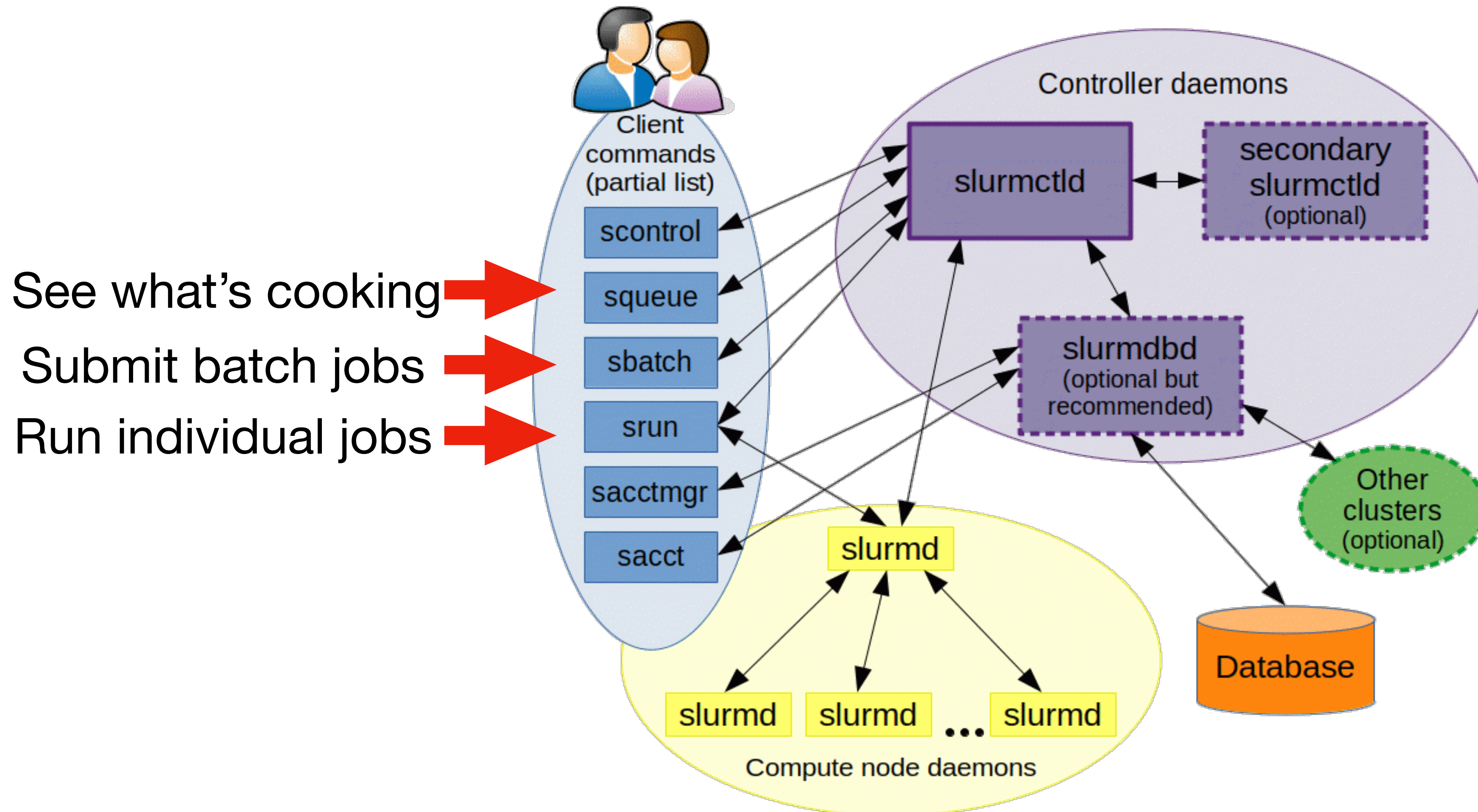
Getting started

(<https://columbiauniversity.atlassian.net/wiki/spaces/rcs/pages/62141892/Ginsburg+--+Getting+Started>)

- Log in to cluster through VScode (connect to ssh:)
- Switch to interactive compute node
 - `srun --pty -t0-01:00 -n 4 -A edu /bin/bash`
 - Better yet: edit your `.bashrc` and add
 - `alias srini='srun --pty -t0-01:00 -n 4 -A edu /bin/bash'`

Introduction to Slurm

<https://slurm.schedmd.com/quickstart.html>



Introduction to Modules

- Many clusters use modules (as do I on my laptop)
- Modules control your environment and available software and let you load and unload packages easily
- Key commands
 - module avail (see what modules are available)
 - module load xxx
 - module unload xxx
 - module purge
 - module search <string> (older systems try module spider or module keyword)

Introduction to Modules

- You can make your own modules.
- Add to your .bashrc
 - `export MODULEPATH=~/modules:${MODULEPATH}`
- Put your own module files in the directory ~modules
- E.g. a custom petsc build for 3.24.4

```
#!/Module

proc ModulesHelp { } {
    puts stderr "This module sets the path and environment variables for petsc-3.24.4"
    puts stderr "    see https://petsc.org/ for more information    "
    puts stderr ""
}

module-whatis "PETSc - Portable, Extensible Toolkit for Scientific Computation"

module load anaconda/3-2022.05 gcc/11.2.0 openmpi/gcc/64/4.1.7a1 intel-oneAPI-toolkit/mkl

set petsc_dir  "/burg-archive/home/mws6/repos/gitlab/petsc"
set petsc_arch "apma4302-pkgs-opt"

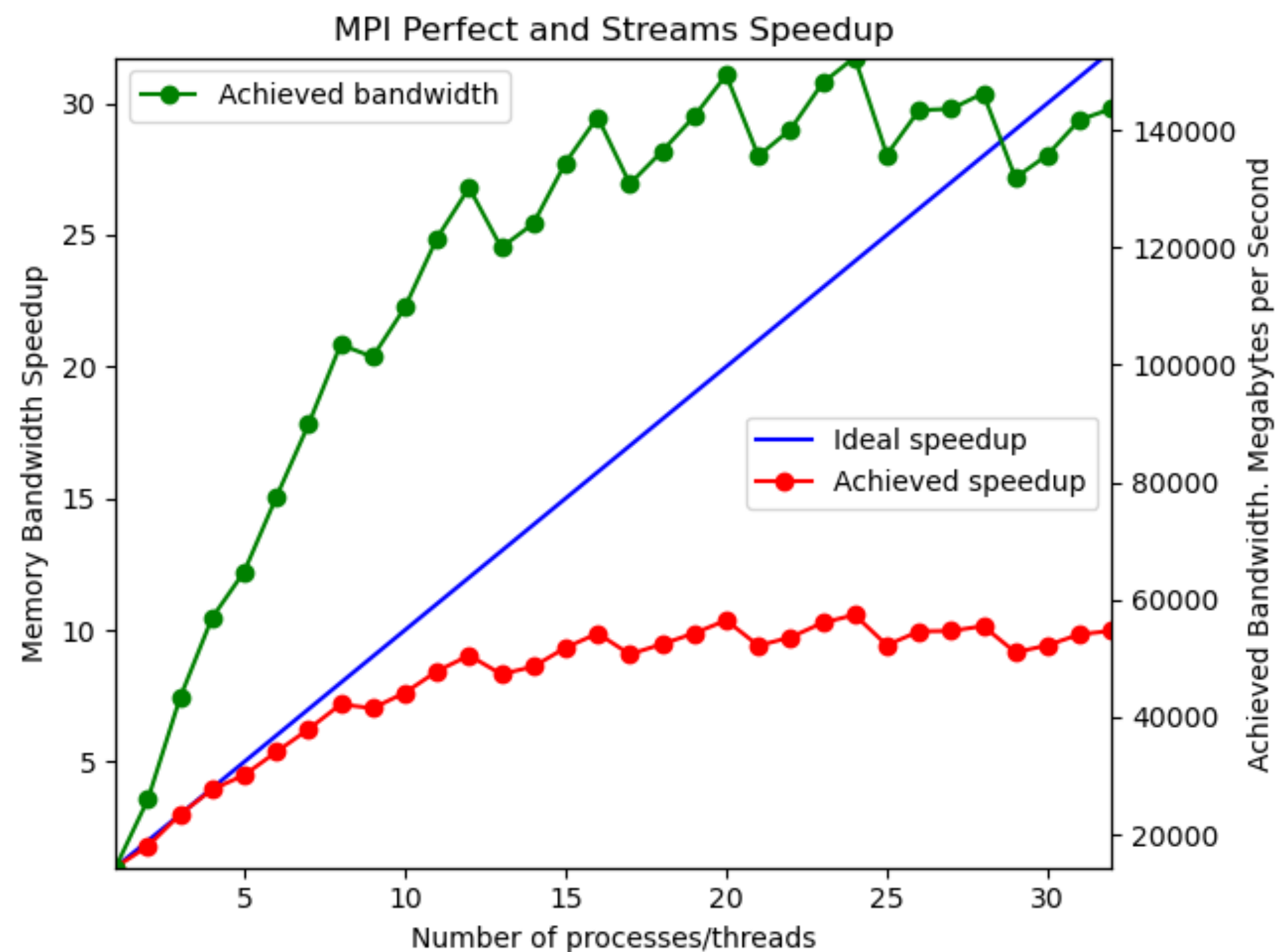
setenv PETSC_ARCH "$petsc_arch"
setenv PETSC_DIR "$petsc_dir"
prepend-path PATH "/burg-archive/home/mws6/repos/gitlab/petsc/apma4302-pkgs-opt/bin:/burg-archive/home/mws6/repos/gitlab/petsc/bin"
```

An example: on ginsburg

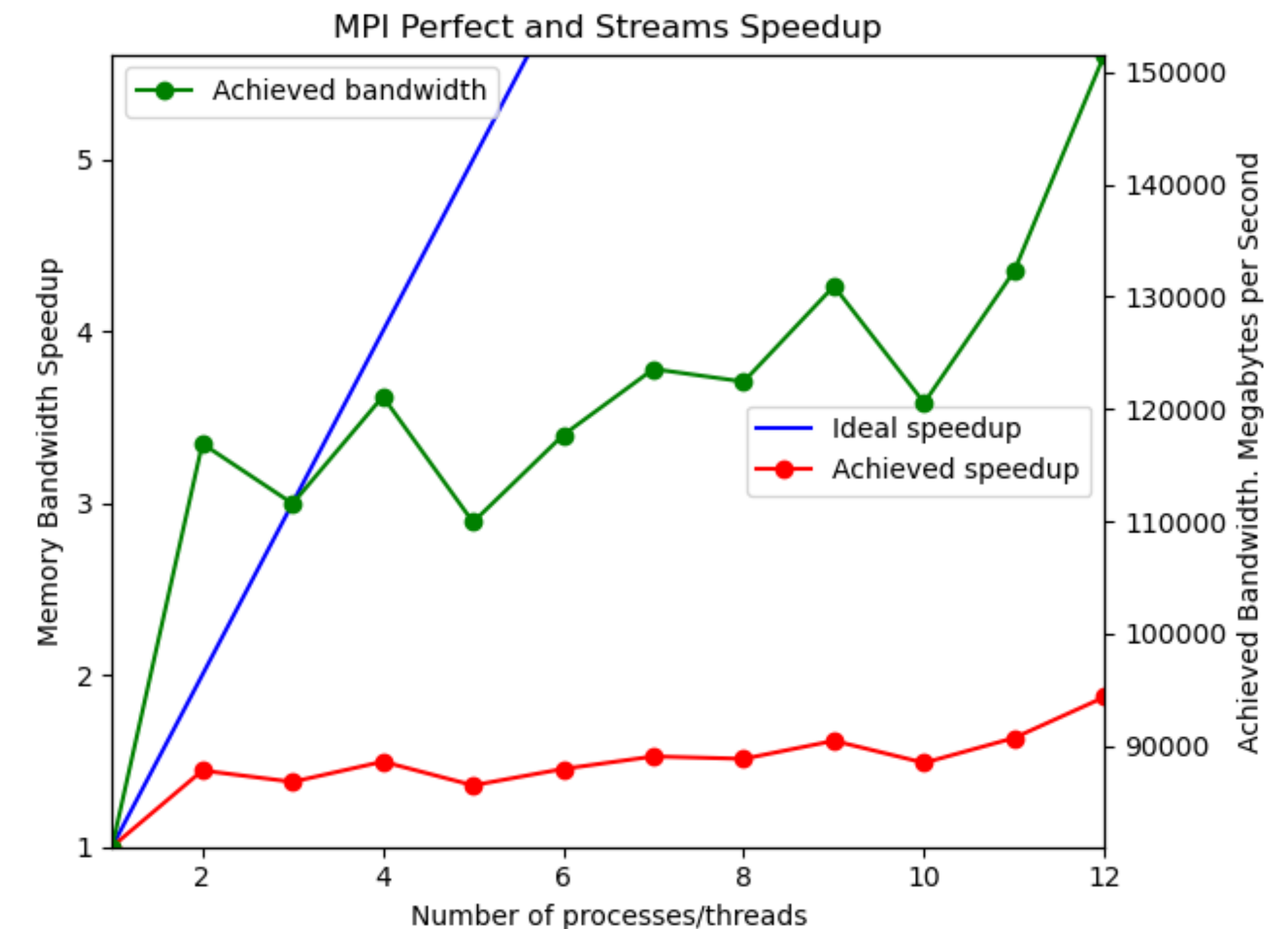
- module load petsc/3.24.4
- module list
- cd \$PETSC_DIR
- make streams

Measuring memory Bandwidth (streams)

- low-arithmetic exercise to measure memory bandwidth for loading and adding two vectors (basic operation in numerical PDE's). Built into petsc distribution.



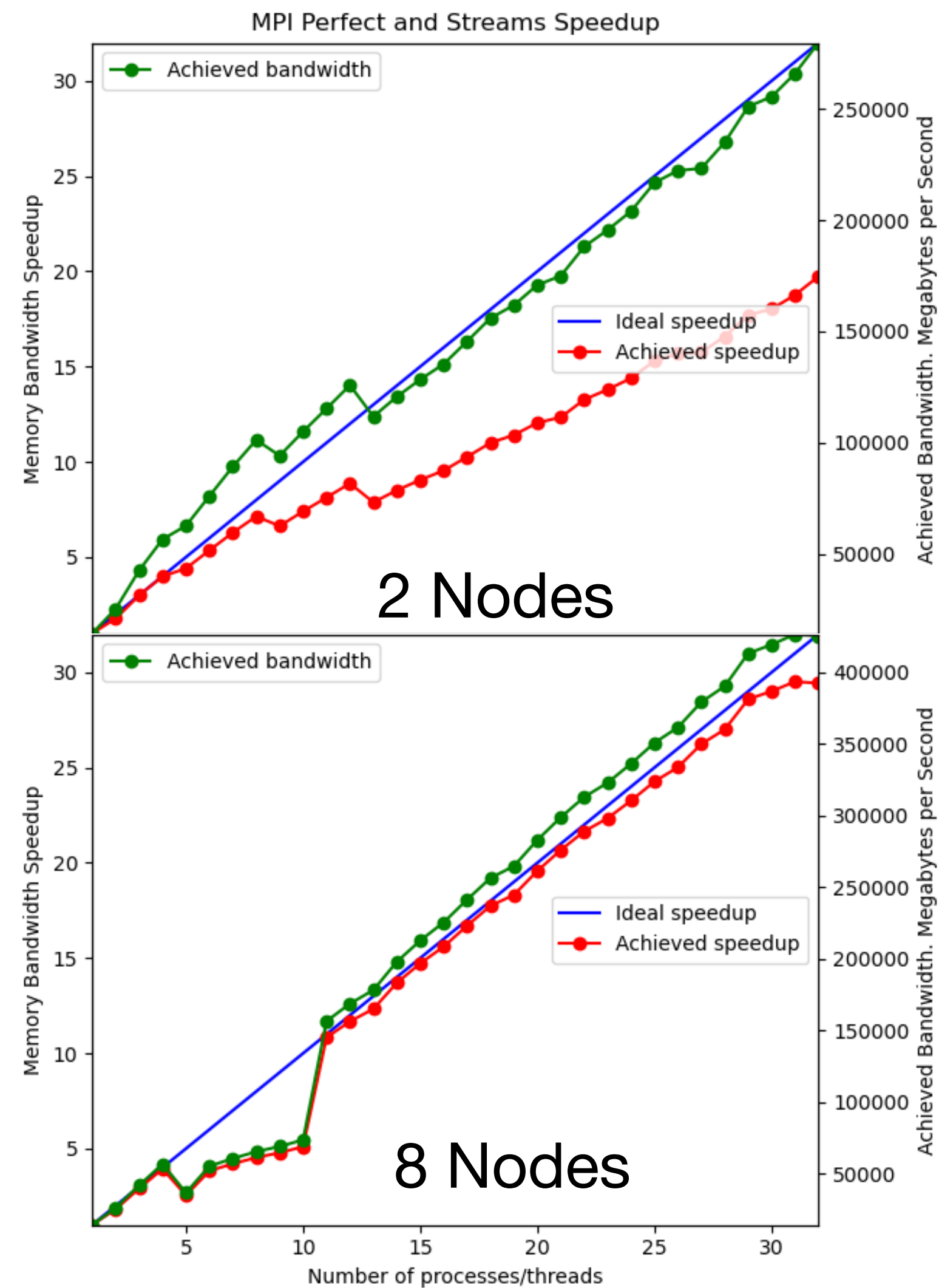
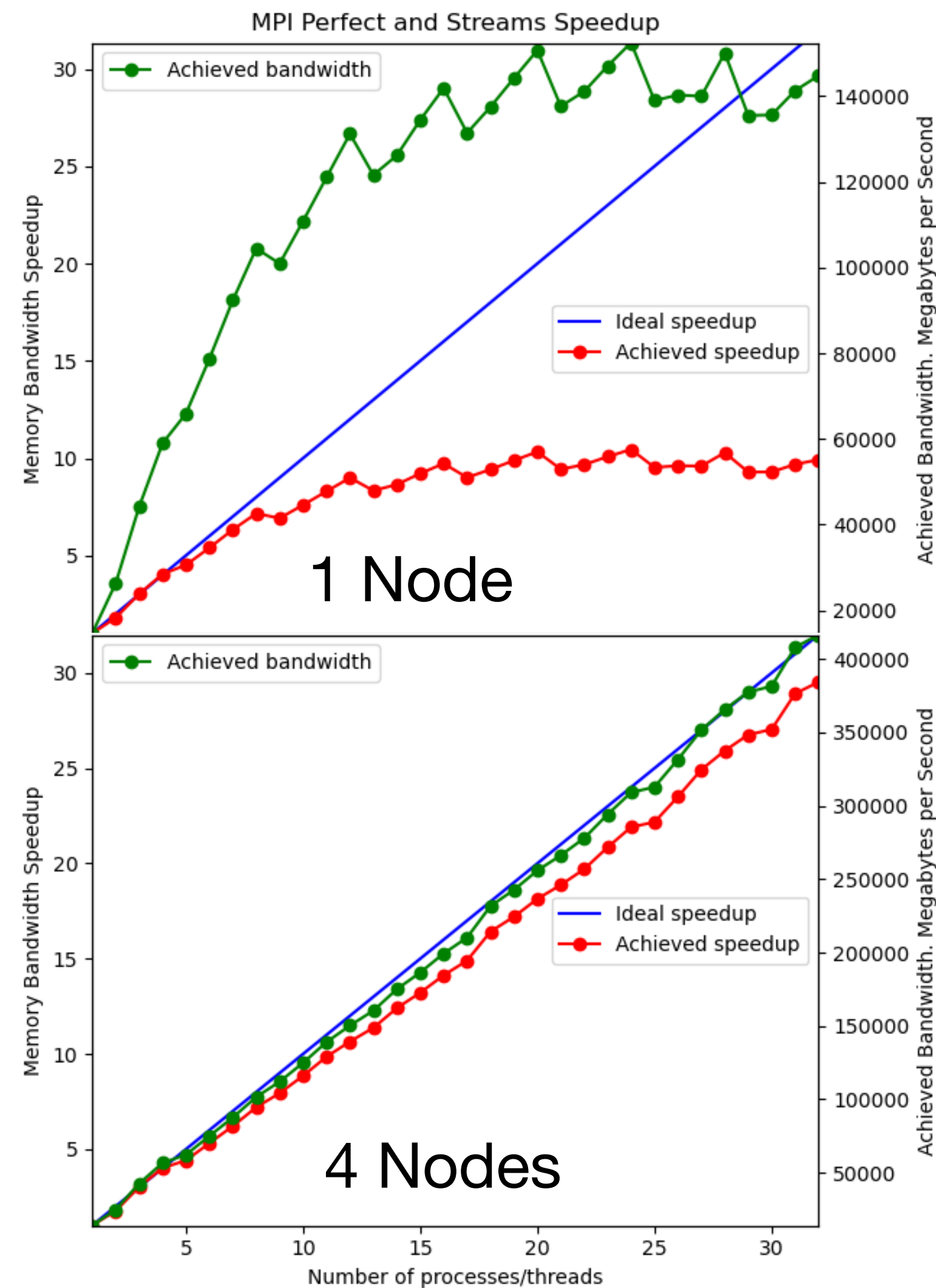
Streams Benchmark Ginsburg



Streams Benchmark Apple M2

Measuring memory Bandwidth (streams)

Streams performance for 32 cores distributed over N nodes: Ginsburg



Time to actually run something

The issues

- You've developed something lovely in petsc or firedrake on your laptop
- You want to move it to a cluster
- The cluster you have access to doesn't have a compatible (or any) petsc or firedrake
- Options:
 - Try to build it yourself given available modules
 - Beg the cluster sysadmin to do it for you (or at least help)
 - Use **containers** (docker/apptainer)

Containers:

Localized virtual machines

- Can run any OS on any other OS.
- Can just package small pieces of code
- Or can package large complex software stack (like firedrake)
- Are composable and portable
- protect against version creep
- Two major flavors we will use
 - On laptops: Docker Desktop
 - On Clusters: Apptainer (which can wrap docker images)

Docker

<https://www.docker.com/>

- Download Docker Desktop
- Pull the latest firedrake container (see <https://www.firedrakeproject.org/firedrake/install.html#docker>)
 - `docker pull firedrakeproject/firedrake:latest`
 - `docker pull firedrakeproject/firedrake-vanilla-default:latest` (smaller version)
- Run the docker image
 - `docker run -it firedrakeproject/firedrake:latest`
 - you can also attach to a running docker image using `vscode`
- Caution: this will drop you into the container as root

Building/adding to Docker Containers

The Docker File

- Describes how to build the software stack on some architecture
- see the directory docker in the firedrake or firedrake-ts repo
- Docker files are composable, can build on previous versions: example `dockerfile.firedrake-ts`

```
solving_utils.py  cahn-hilliard.py M  Dockerfile.firedrake-ts firedrake_ts/... U  Dockerfile.firedrake-ts docker >
docker > Dockerfile.firedrake-ts > ...
You, 2 days ago | 1 author (You)
1  # Dockerfile for Firedrake with a full set of capabilities installed.
2
3  FROM firedrakeproject/firedrake:latest
4
5  # install fixed branch of firedrake-ts
6  ARG BRANCH="2026.4-fixes"
7
8  RUN git clone --branch ${BRANCH} https://github.com/mspieg/firedrake-ts.git /opt/firedrake-ts \
9  |    && pip install --verbose /opt/firedrake-ts
10
```

Building/adding to Docker Containers

The Docker File

- Building a new image
- run
 - `docker build --platform linux/amd64 -t mspieg/firedrake_ts_amd:latest --push .`
- Can now pull from docker hub.
 - `docker pull mspieg/firedrake_ts_amd:latest` (for x86 architecture)
 - `docker pull mspieg/firedrake_ts_amr:latest` (for macOS ARM)
- And run as interactive shell with current directory bound to `\tmp`
 - `docker run -it -v `pwd`:/tmp mspieg/firedrake-ts_arm:latest`
- Once a container is open, you should be able to connect to it using viscode but files are an issue.

Containers on Clusters: Apptainer

- Docker images will also run on clusters but a more useful approach is to use *apptainer* (formally singularity)
- Apptainer:
 - more portable: puts containers into one movable .sif file
 - Can create sif file directly from docker containers
 - `apptainer build firedrake-ts.sif docker://mspieg/firedrake-ts amd:latest`
- Can run interactive shell on your own filesystem easily
 - `apptainer shell firedrake-ts.sif` (example)
- Can also run in exec mode or with `mpiexec`
- DEMO on Ginsburg

Containers on Clusters: Apptainer

Cheat sheet (can demo on insomnia)

- Create an apptainer .sif file from a docker container
 - `apptainer build firedrake-ts.sif docker://mspieg/firedrake-ts_amd:latest`
- Run container in an interactive shell:
 - (you might need to set some environment variables first)
 - `apptainer shell firedrake-ts.sif`
- execute a command from within a container (module load mpi)
 - `apptainer exec <path-to>/firedrake-ts.sif mpiexec -np 2 ./fish`
 - `mpiexec -np 2 apptainer exec <path-to>/firedrake-ts.sif ./fish`
 - `mpiexec -np 2 apptainer exec <path-to>/firedrake-ts.sif python3 helmholtz_snes.py`
 - `mpiexec -np 2 apptainer exec <path-to>/firedrake-ts.sif python3 heat.py`
- wrap the whole thing in a slurm script and submit using sbatch

Batch jobs in slurm: using sbatch

Sample slurm script to do strong scaling on 3D poisson problem

```
#!/bin/bash
#SBATCH --account=apam
#SBATCH --exclusive
#SBATCH --ntasks=64
#SBATCH --tasks-per-node=8
#SBATCH --time=120
#SBATCH --mail-user=USERNAME@columbia.edu
#SBATCH --output=strong_fish_N257_P064_p08.o.%j

# set some environment variables for apptainer
module load openmpi/gcc/64/4.1.7a1
module load singularity
export APPTAINER_TMPDIR=$SINGULARITY_TMPDIR
export APPTAINER_BINDPATH=$SINGULARITY_BINDPATH

# set the container for firedrake
SIF=/burg/home/mws6/sifs/firedrake-ts.sif

# Launches the MPI application.
GO="mpiexec -n $SLURM_NTASKS apptainer exec $SIF"
cd $SLURM_SUBMIT_DIR

# FISH: solve 3D Poisson equation
# using optimal CG+GMG solver
# -da_refine 7 is 257x257x257
# ~ 12 Gb memory
# coarse grid is 9x9x9 so should work up to several hundred processors
$GO ./fish -fsh_dim 3 -da_refine 7 -pc_mg_levels 6 -pc_type mg -ksp_type cg -snes_type ksponly -ksp_converged_reason -ksp_monitor -log_view
```

Parallel Performance

Basics

- Working Definitions
 - Parallel Speedup
 - Parallel Efficiency
 - Static Scaling
 - Strong Scaling
 - Weak Scaling
 - ‘dyscalia’ plot
- Testing Parallel speedup on Ginsburg for fish.c, fish.py (PETSc and Firedrake) using containers...

Parallel Performance

Two fundamental variables: N, P

- N : problem size (e.g. Degrees of Freedom)
- P : concurrency — number of parallel processors

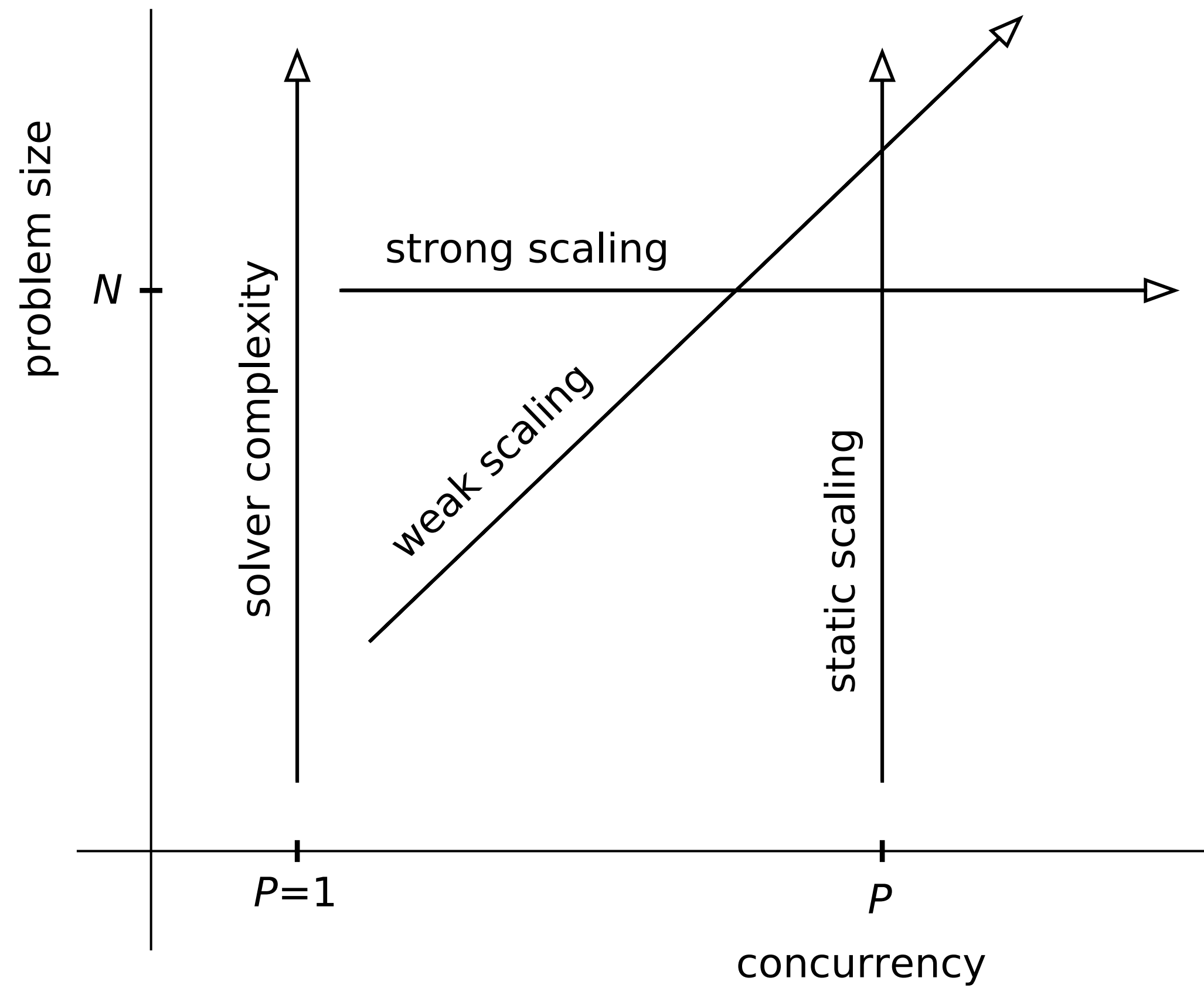
Basic Definitions: if $t(N,1)$ is the time it takes to solve a problem of size N on 1 processor then

– Speedup: $s_N(P) = \frac{t(N,1)}{t(N,P)}$ (perfect speedup $s_N(P) = P$)

– Parallel Efficiency: $e_N(P) = \frac{s_N(P)}{P} \in [0,1]$

Parallel Performance

Classical Definitions of Scaling:



Optimal solver complexity

$$t(N,1) = O(N)$$

Optimal strong scaling

$$s_N(P) = P \text{ for fixed } N$$

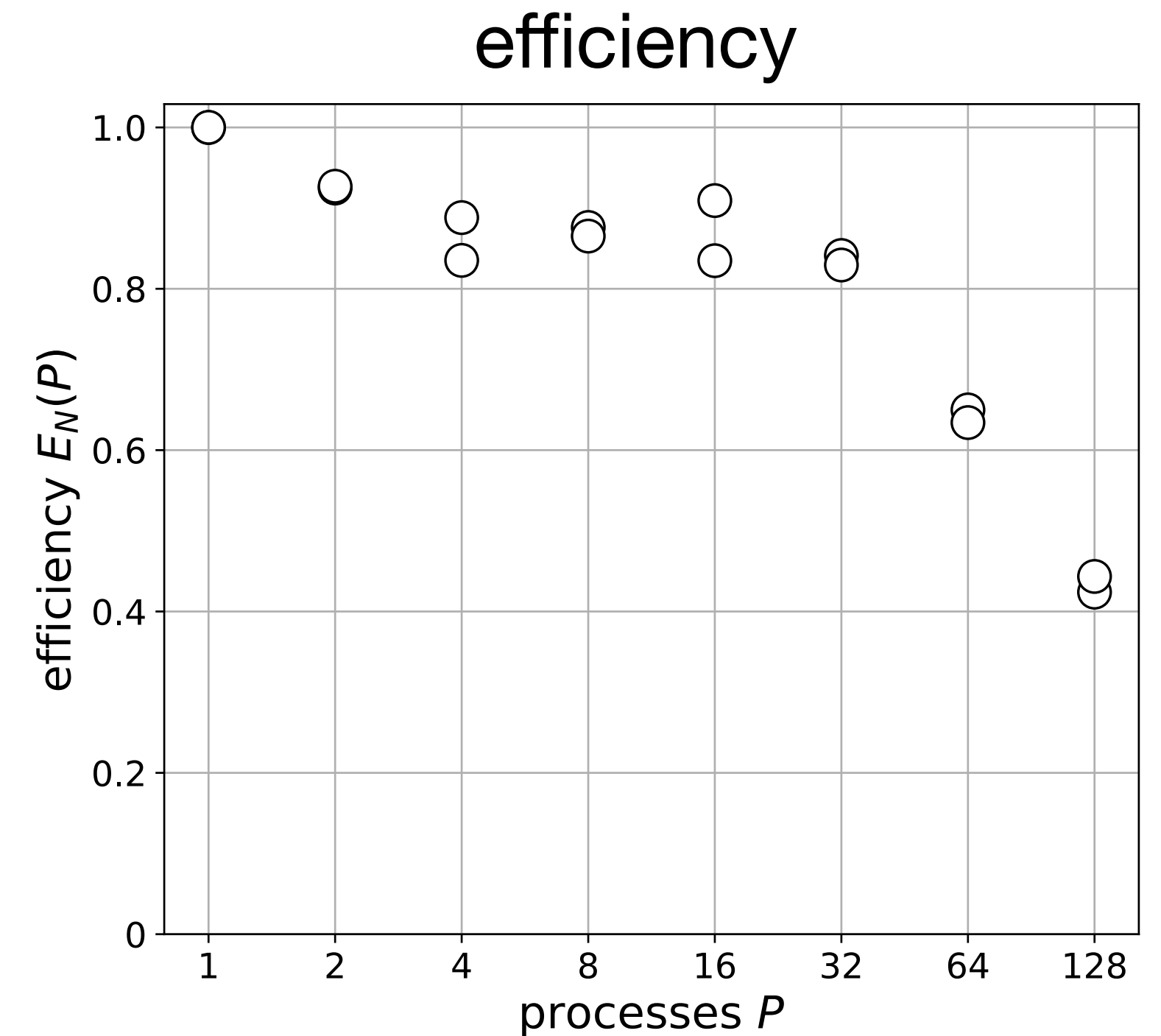
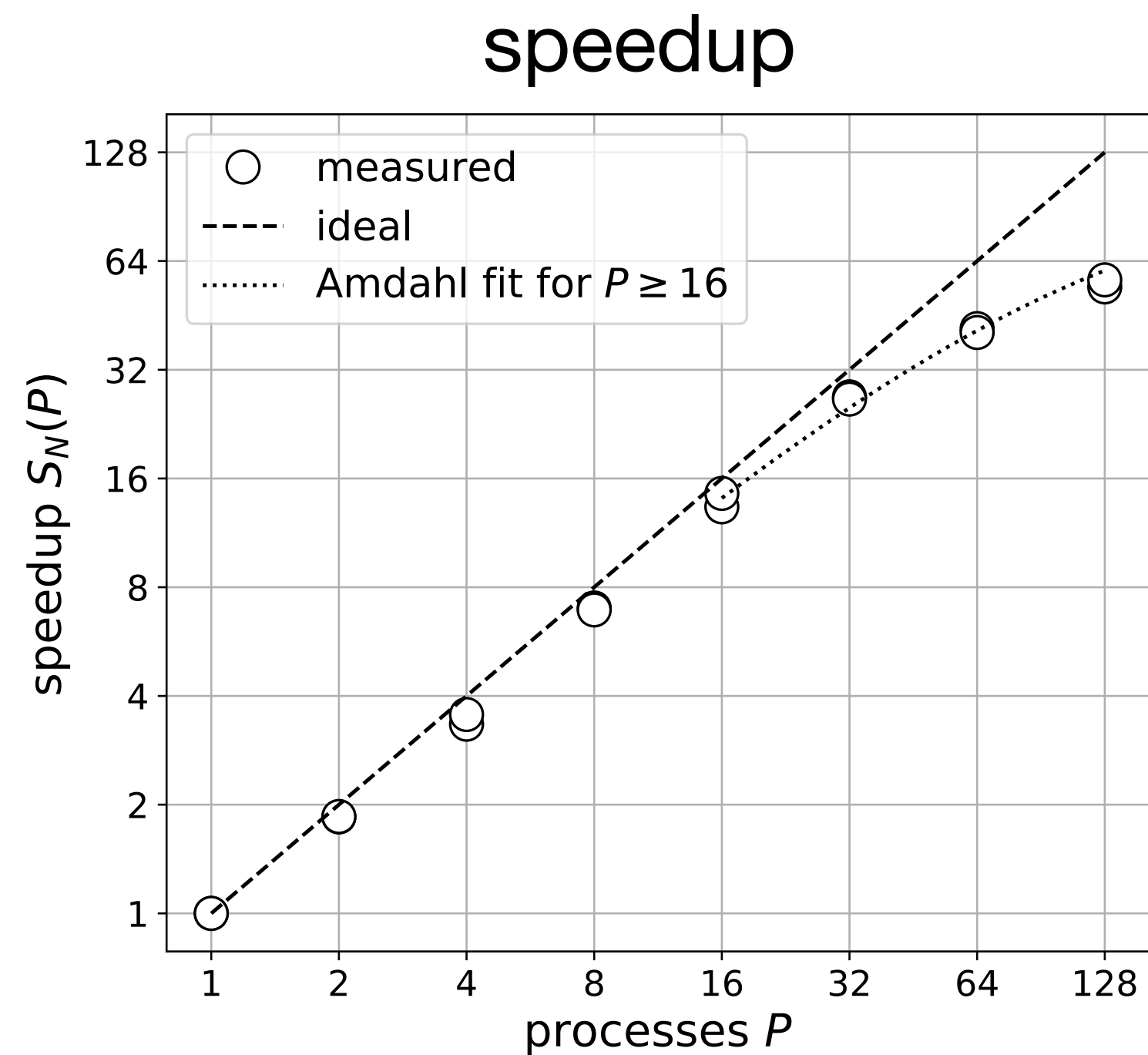
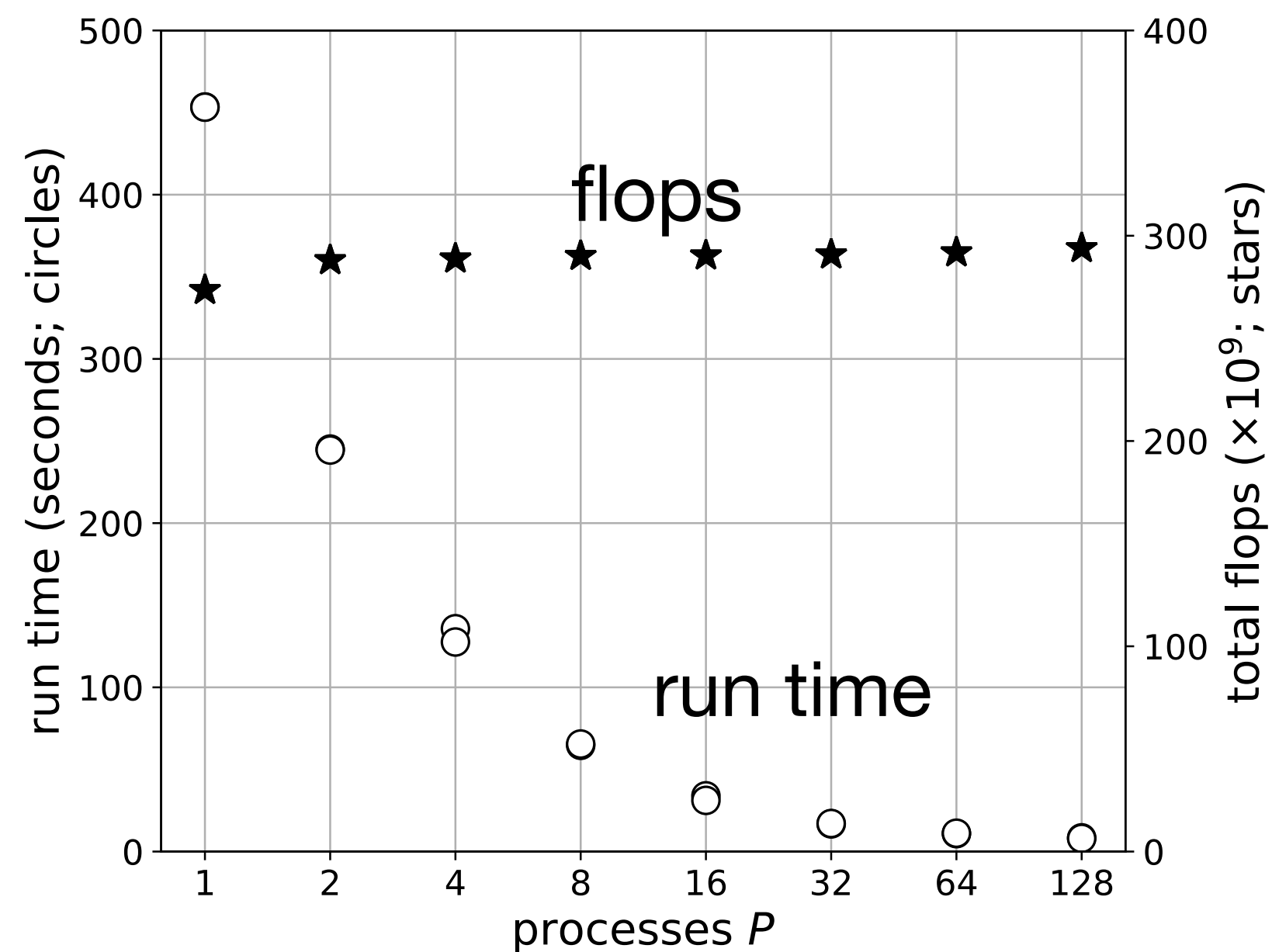
Optimal weak scaling

$$e_N(P) = 1 \text{ for fixed } N/P$$

Figure 8.2. Strong, weak, and static scaling, as well as serial solver complexity, are directions to infinity in the (N, P) plane.

Parallel Performance

Strong Scaling: Example from Bueler ch 8 (minimal: ch 7)



Amdahl's Model: $t(N, P) = f_s T(N, 1) + (1 - f_s) \frac{t(N, 1)}{P}$

or: $s_N(P) = \frac{P}{f_s(P - 1) + 1}$

Parallel Performance

Weak Scaling: Example from Bueler ch 8 (minimal: ch 7)

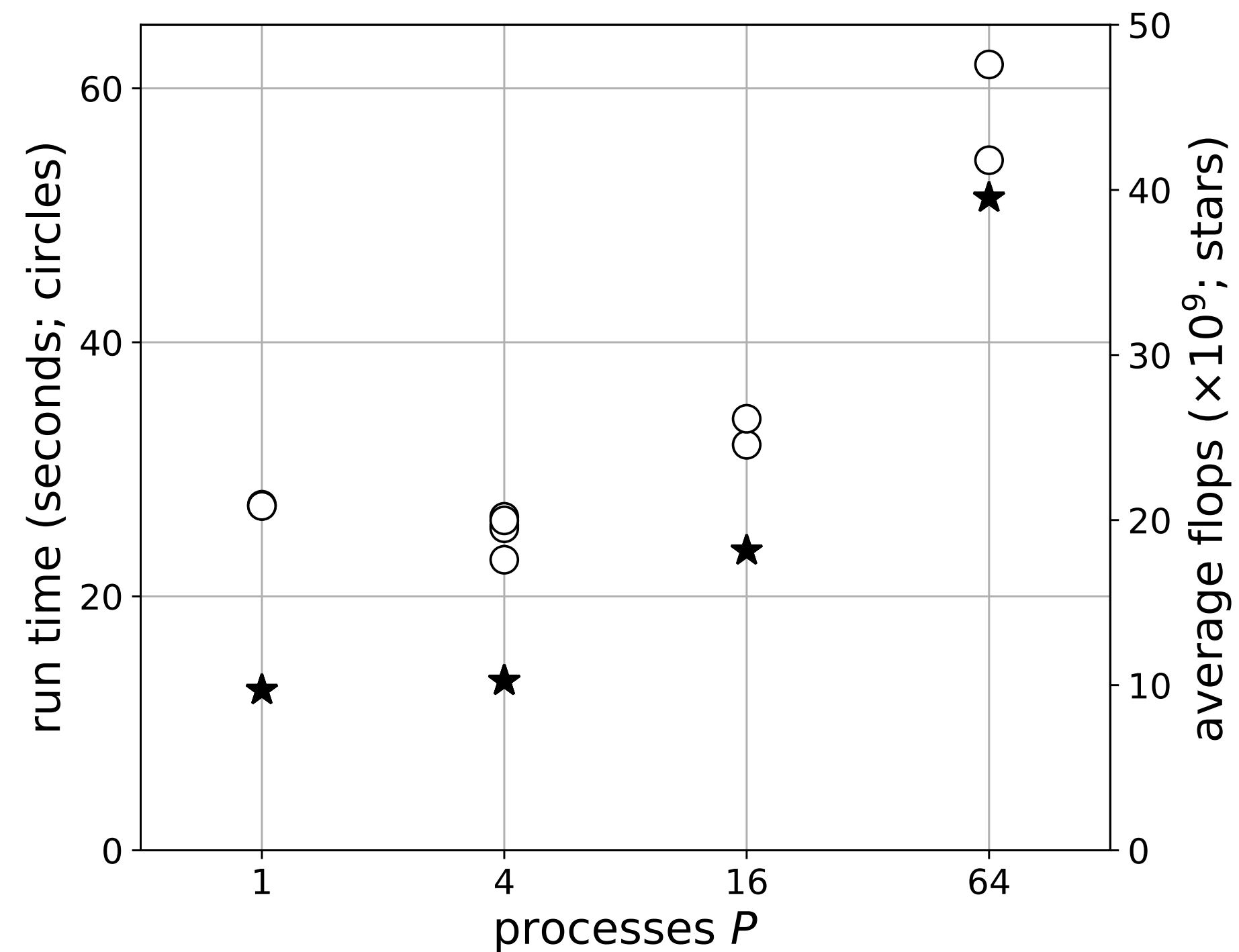
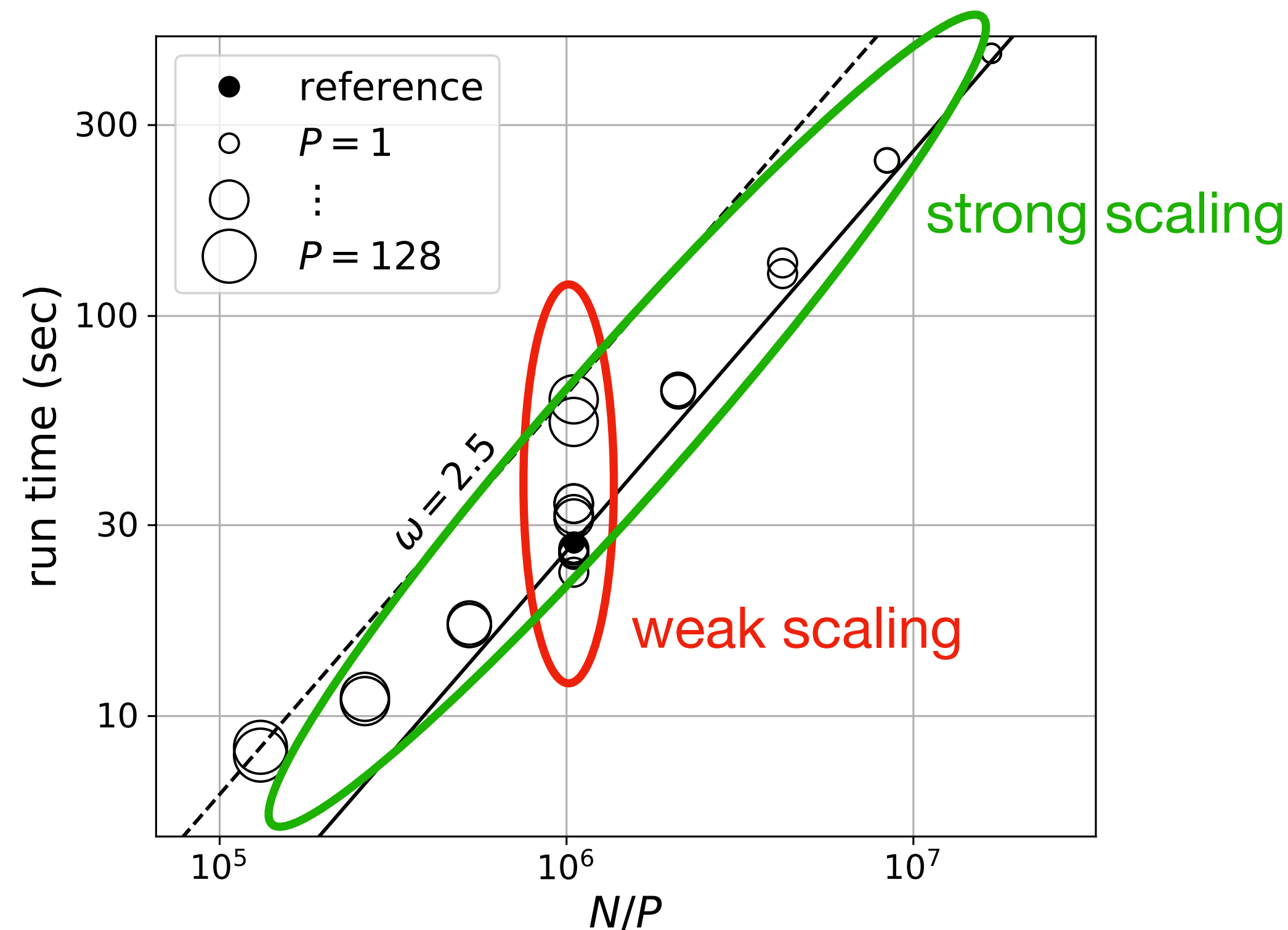


Figure 8.7. Run time $t(N, P)$ and average flops $f(N, P)/P$ in a weak-scaling study of `ch7/minimal.c`.

Parallel Performance

Holistic Scaling and “dyscalia”: plot $t(N, P)$ vs N/P



Ideal Scaling

$$t(N, P) = \gamma \left(\frac{N}{P} \right)$$

def: dyscalia

$$\omega = \max_i \left[\frac{t(N_i, P_i) P_i N_0}{t(N_0, 1) N_i} \right]$$

ratio of total run time per degree of freedom
for run i :

$$P_i t(N_i, P_i) / N_i$$

vs total run-time per DOF for 1 processor

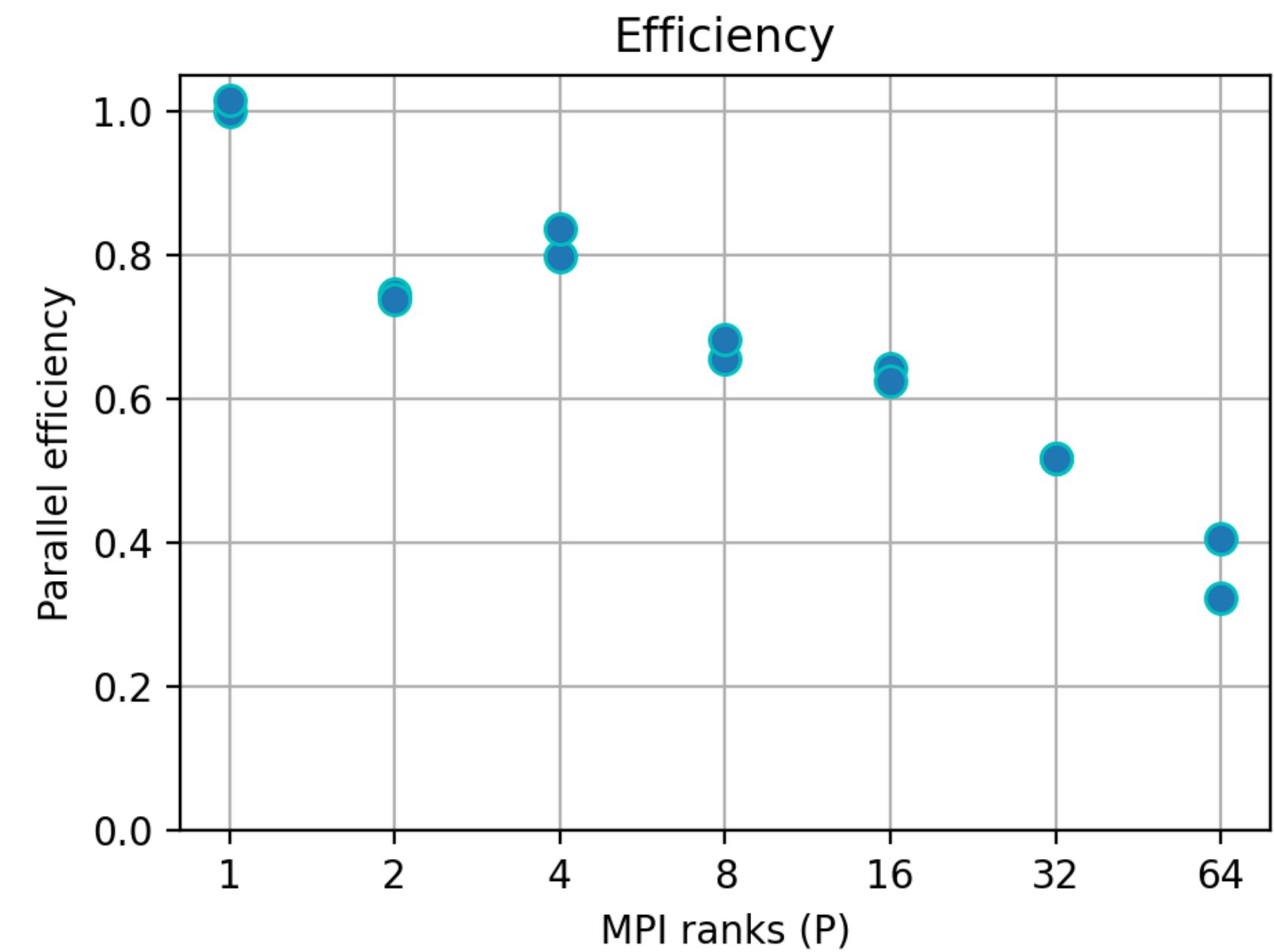
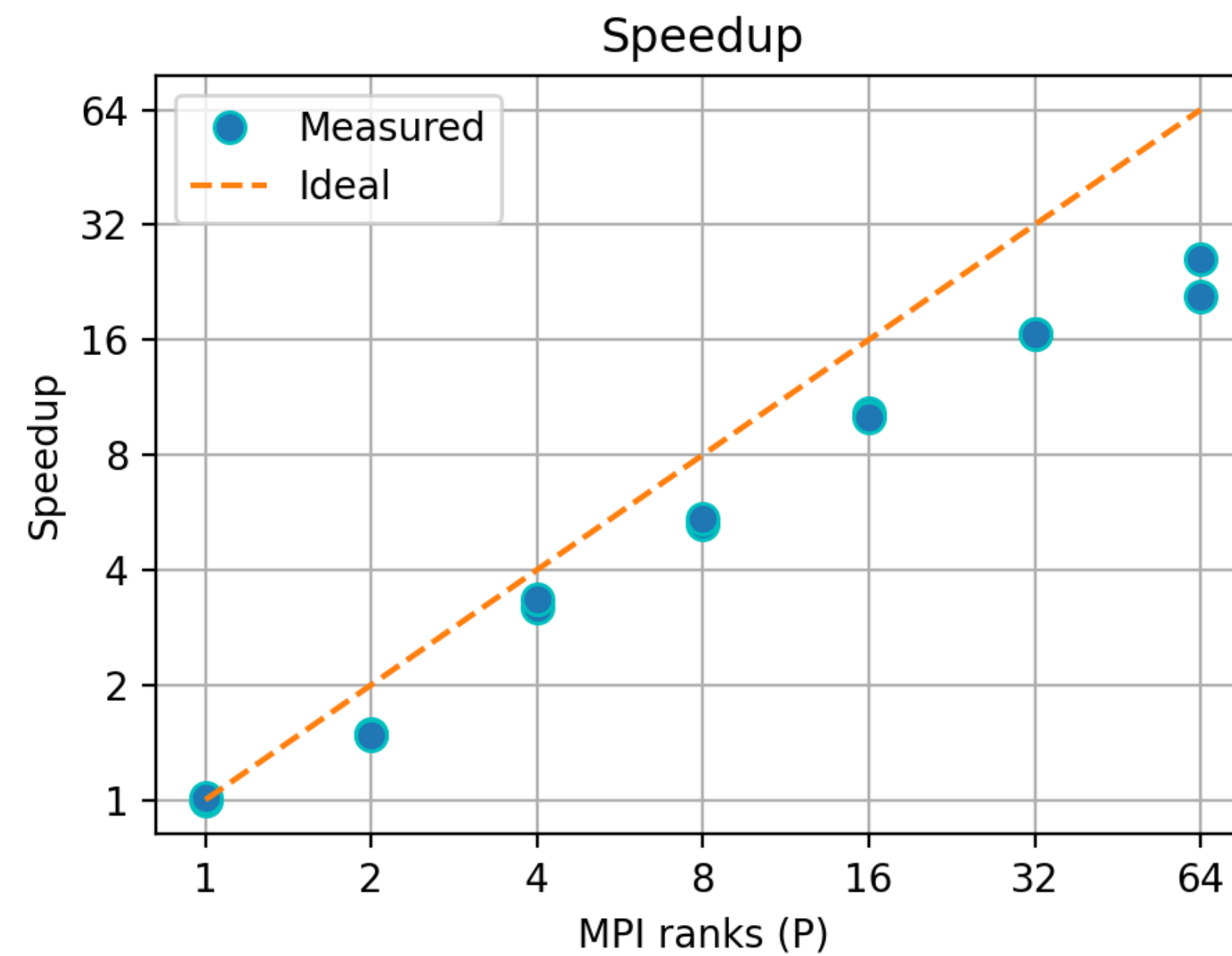
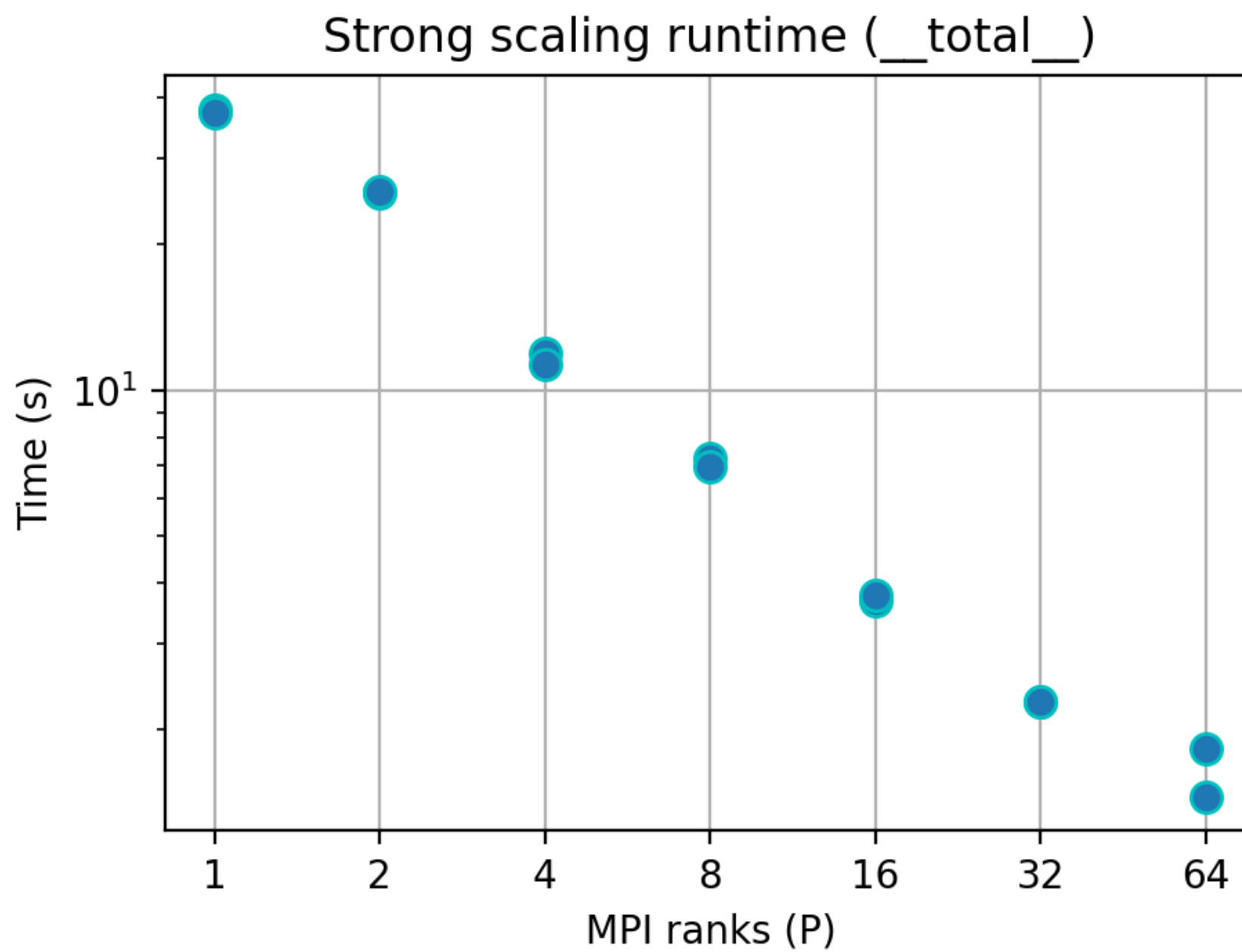
$$t(N_0, 1) / N_0$$

Figure 8.8. A holistic scaling view of scaling for `ch7/minimal.c`: $t(N, P)$ versus N/P on log-log axes. The observed dyscalia $\omega = 2.5$, from (8.3), measures the failure of ideal scaling (8.2).

Parallel Performance

Strong Scaling: Ginsburg 3-D poisson solver 8 cores/node

strong_fish N=257 (DoF=16,974,593)



Parallel Performance

Strong Scaling: Ginsburg 3-D poisson solver 4 cores/node

strong_fish N=257 (DoF=16,974,593)

